

Biblioteca Digital & SimplODB

Desafios de Persistência e Arquitetura Java

Backend

Membros: Lucas Freitas, Adriana Selestrina, Yasmin Serra

Visão Geral do Projeto

Biblioteca Digital

Sistema robusto para gestão de acervo, usuários e empréstimos, focado em regras de negócio complexas e integridade de dados.

SimploDB

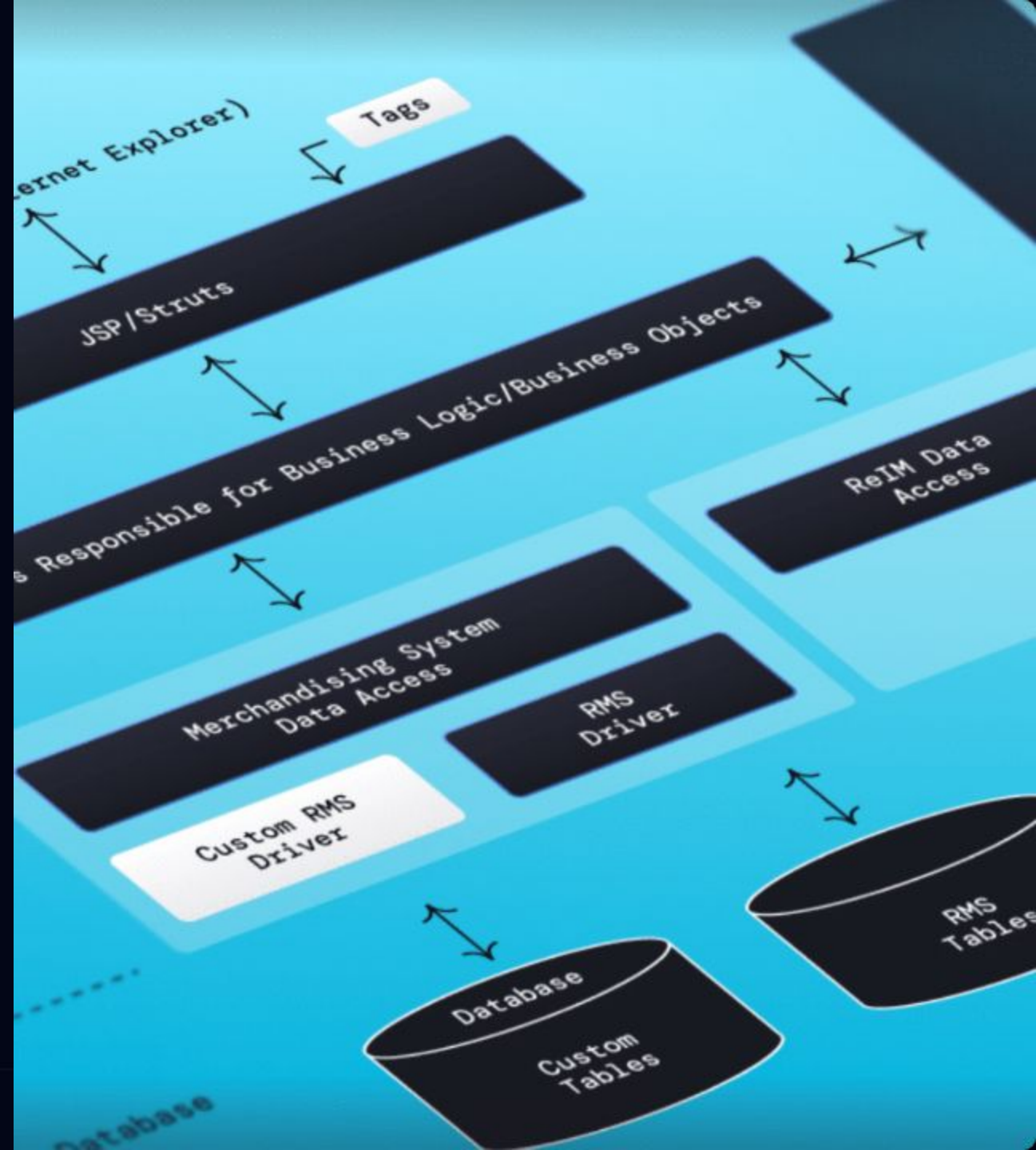
Uma solução customizada de banco de dados que utiliza persistência binária para garantir que nenhuma informação seja perdida entre execuções.

Arquitetura: Fluxo do Dado

Desde a chamada do método salvar() até a escrita física no disco:

- **Mapeamento:** Definição da rota db/{entidade}/{id}.dat no arquivo motor.
- **Transformação:** Serialização do objeto Java em bytes.
- **Escrita:** Uso de streams de saída para gravar o arquivo binário.

Analogia: É o ato de etiquetar um livro e guardá-lo em uma prateleira específica na despensa.



Serialização: Vantagens e Limites



Vantagens

"Fidelidade Total" (O objeto é salvo exatamente como é).



Limitações

"A Busca é por Varredura" (Não temos índices).

"Dependência do Java" (Só o Java entende esses arquivos).

Analogia: É como procurar uma palavra em um livro que não tem sumário nem índice. Você é obrigado a ler página por página até encontrar o que quer. Para 100 livros funciona bem, para 1 milhão, o sistema ficaria lento.

O "Pulo do Gato" Arquitetural

Generics & Herança

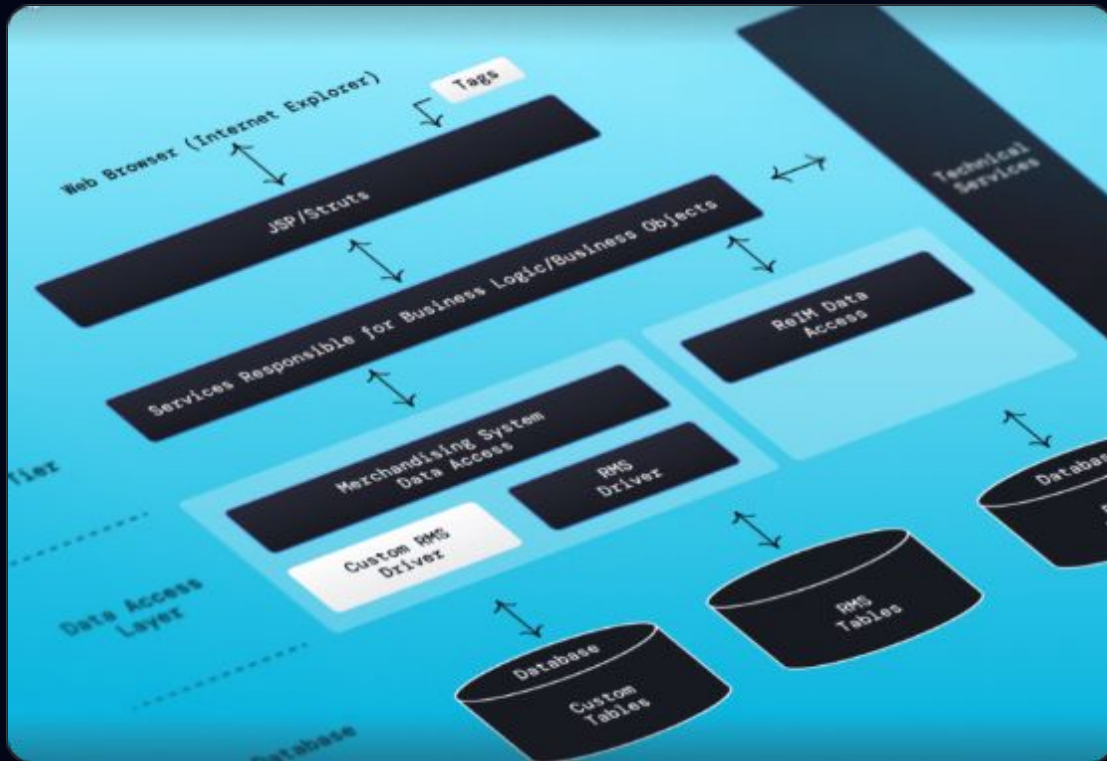
A parte mais interessante foi a implementação dos métodos sobre a arquitetura `Repositorio<T>`.

O professor nos forneceu a base do motor, e o mais interessante foi implementar os métodos de forma que a lógica de persistência fosse totalmente separada da lógica de negócio. Ao utilizarmos o padrão `Repositorio<T>`, garantimos que a nossa implementação no `LivroRepositorio` fosse limpa.

*Com isso, não precisamos escrever um código para salvar 'Livros' e outro para 'Usuários'. Escrevemos uma lógica que entende o Contrato de Persistência. Isso é o que se conhece como código extensível: se amanhã a biblioteca quiser alugar 'Computadores' ou 'Celulares', o sistema está pronto sem mudar uma linha da base que construímos.

Analogia: Criamos uma lógica que aceita qualquer "plugue" de dado, tornando o sistema infinitamente escalável para novas entidades.

JAVA STREAMS API



O Filtro - Java Streams API

Utilizamos Streams para processar listas de livros e gerar relatórios dinâmicos.

```
.stream()  
.filter(l -> l.getGenero() ...)  
.sorted(Comparator.reversed())  
.collect(Collectors.toList())
```

Analogia: Como um filtro de café: entra o pó (dados brutos) e sai apenas o líquido puro (o resultado refinado).

API DE DATAS

Gestão Temporal

Utilizamos LocalDateTime para precisão de segundos e ChronoUnit para lógica de multas.

```
long dias = ChronoUnit.DAYS.between(  
    dataDevolucaoPrevista, referencia  
);
```

Analogia: O Relógio de Ponto Como um cartão de ponto em uma fábrica: ele registra o segundo exato da entrada e saída para calcular horas extras (ou multas).

Por que é importante?

- Cálculo preciso de **R\$ 1,00/dia** de multa.
- Uso de isAfter() para verificar atrasos sem erros de "off-by-one".
- Formatação amigável com DateTimeFormatter no toString().

FUNCIONAL & PREDICATE

O Poder do Predicate

O método `buscarComFiltro` é o coração das consultas do SimploDB.

Analogia: O Controle de Qualidade Em vez de criar uma regra para cada item, você dá ao inspetor (Predicate) uma lupa e uma instrução: "Deixe passar só o que for Romance".

```
public List buscarComFiltro(  
    Predicate filtro  
) {  
    return buscarTodos().stream()  
        .filter(filtro)  
        .collect(Collectors.toList());  
}
```

Decisão Técnica: Flexibilidade total. Podemos filtrar por ISBN, Nome ou Ano sem mudar o motor.

Dificuldades e Desafios

-  **Versionamento de Código:** Gerenciar conflitos de merge e branches paralelas exigiu disciplina rigorosa no Git.
-  **Realização de Testes:** Tivemos que desenvolver testes "mock" para validar cada requisito isoladamente sem depender do arquivo físico.
-  **Reorganização de Equipe:** A saída de um membro na reta final exigiu um esforço de todos para redistribuir tarefas críticas e garantir a entrega do projeto..

Principais Aprendizados



Comunicação Assertiva

- Aprendemos que a comunicação clara entre os membros é tão importante quanto a qualidade do código fonte.
- O reforço nos conteúdos do módulo (Streams, Optional, NIO.2) nos deu confiança para entrega do projeto e arquitetar sistemas reais.

Demo ao Vivo

Executando o Main.java: Salvamento, Listagem e Filtros em ação.



Vejamos os bits se transformarem em livros!